

CK3 Character History Generator

Comprehensive Web Architecture & Technical Design Document

Prepared for: LLM Code Generation (Claude Max)

This document contains the complete technical specifications, data models, parser algorithms, and output formatting rules required to build the web-based simulation engine. It serves as a strict AMENDMENT to the existing React/FastAPI codebase. Modify the existing code to accommodate these refined extraction rules and mechanics.

May 4, 2026

Contents

1	Scope of Amendment	2
1.1	1.1 Context	2
1.2	1.2 Targeted Refactoring Goals	2
2	UI/UX & State Amendments	3
2.1	2.1 Redux Store Updates	3
2.2	2.2 UI Refinements	4
3	Aggressive AST Parser Amendments	5
3.1	3.1 Parsing Landed & Landless Titles	5
3.2	3.2 Parsing Religions & Faiths	5
3.3	3.3 Parsing Governments	5
3.4	3.4 Parsing Trait Incompatibilities	6
3.5	3.5 Title History Decompilation	6
4	Simulation Engine Amendments	7
4.1	4.1 AI Weighting (Marriage & Murder)	7
4.2	4.2 Relationships & Secrets	7
4.3	4.3 Cadet Branch Splintering	7
5	Output Formatting Overrides	8
5.1	5.1 Character History Format	8
5.2	5.2 Title History Formats	9
5.3	5.3 Dynasty & Cadet House Output	9

Chapter 1

Scope of Amendment

1.1 1.1 Context

The current application successfully handles basic drag-and-drop file parsing, Redux state management, and Celery-queued simulation generation. To support total conversion mods with advanced mechanics, the engine must be expanded.

1.2 1.2 Targeted Refactoring Goals

- **Data I/O Expansion:** The upload dropzones must accept *arrays* of files (e.g., multiple trait, title, and government files). The AST parser must be heavily optimized to discard visual/map bloat and extract only functional keys.
- **Decompilation:** The engine must support reverse-engineering (decompiling) existing `title_history.txt` files back into the visual sequence builder.
- **Simulation Depth:** Upgrading the Python engine to support Cadet Branch splintering, AI weighting for marriages/murders, Trait incompatibilities, Secrets, and inter-character Relationships.
- **Out of Scope:** Character memories, culling algorithms, and 3D DNA strings are strictly omitted. Do not implement them.

Chapter 2

UI/UX & State Amendments

2.1 2.1 Redux Store Updates

The `RootState` must be expanded to include the new toggleable features, government arrays, and decompiled title data.

```
interface RootState {
  globalSettings: {
    startYear: number;
    endYear: number;
    ignoreTitleGeneration: boolean; // Skip titles, gen characters only
    enableSecrets: boolean;
    enableRelationships: boolean;
  };
  dynasties: Array<{
    id: string;
    name: string;
    baseCulture: string;
    baseReligion: string;
    lowbornSpousesOnly: boolean; // Forces marriage to unlanded generated
    lowborns
  conversions: Array<{ year: number, newCulture?: string, newReligion?:
    string }>;
  }>;
  parsedFiles: {
    titles: Record<string, any>;
    traits: Record<string, any>;
    deathReasons: Record<string, any>;
    religions: Record<string, any>;
    governments: string[]; // Array of parsed government keys
    nameLists: Record<string, string[]>;
  };
  titleSequences: Record<string, TitleSequence[]>;
}

interface TitleSequence {
  dynastyId: string;
  duration: number;
  transitionType: "marriage" | "usurpation" | "extinction";
  governmentType?: string; // Selected from parsedFiles.governments
  liegeTitleId?: string; // Must be a higher tier title
}
```

2.2 2.2 UI Refinements

1. **Multi-File Uploads:** Refactor the `FileDropzone` components to accept **multiple** files. The backend API must iterate over the uploaded files and merge the resulting ASTs into single dictionaries.
2. **Gantt Chart for Conversions:** In the Dynasties tab, implement a horizontal Gantt-style timeline. Users can drop a "Conversion Node" at a specific simulation year to schedule a cultural or religious shift for a specific dynasty.
3. **Interactive Node Editing:** In the Visualizer tab, clicking a rendered character node in the family tree must open a modal. This modal allows the user to manually edit the character's traits, death date, or relationships *before* the final ZIP download is generated.

Chapter 3

Aggressive AST Parser Amendments

The Python AST parser must be heavily optimized to prevent memory bloat. Paradox files contain massive amounts of UI/Map data that the simulation does not need. The parser must extract **only** the necessary structural keys.

3.1 3.1 Parsing Landed & Landless Titles

When reading `'00landedtitles.txt'`, the parser must discard cosmetic blocks.

Capture Strategy: Only capture keys matching map prefixes (`e_`, `k_`, `d_`, `c_`, `b_`) or the hegemony prefix (`h_`).

Discard Strategy: Completely ignore and drop blocks named `color`, `capital`, `cultural_names`, `province`, etc.

Landless Titles (LAAMP & NF): Keys prefixed with `d_laamp_` represent wandering bands. Keys prefixed with `d_nf_` represent administrative mansions. The parser must capture these and flag them internally as `landless = yes`.

3.2 3.2 Parsing Religions & Faiths

The parser must extract hierarchical marital doctrines.

- **Religion Block:** Locate the root key (ending in `_religion`, e.g., `adunai_religion`). Extract any marital doctrines (e.g., `doctrine = doctrine_monogamy, doctrine_polygamy, doctrine_concubines`).
- **Faith Block:** Locate the `faiths = { }` sub-block. Extract the individual faith keys (e.g., `faith_kings_men_numenorean`).
- **Inheritance:** Scan the faith for its own marital doctrine. If present, it overrides the religion's doctrine. If absent, the faith inherits the marital doctrine of the parent religion. Discard holy sites, localization, and icons.

3.3 3.3 Parsing Governments

Users will upload files defining government types. The parser only needs the top-level keys.

- **Execution:** Scan the file for root-level keys (e.g., `feudal_government`, `republic_government`, `theocracy_government`).
- **Discard:** Drop all internal contents of the block (`government_rules`, `valid_holdings`, `flags`). Store the keys as an array to populate the frontend dropdown menus.

3.4 3.4 Parsing Trait Incompatibilities

When parsing traits flagged with `genetic = yes` or `category = fame`, the parser must explicitly capture the `opposites = { }` block. This array of incompatible trait IDs must be stored so the genetics and relationship engines prevent characters from holding mutually exclusive traits.

3.5 3.5 Title History Decompile

Create a new POST `/api/decompile/titles` endpoint.

- If a user uploads an existing `title_history.txt`, the parser reads the chronological date blocks inside each title (e.g., `6800.1.1 = { }`).
- It extracts the `holder`, `government`, and `liege` fields.
- It converts this data back into the frontend's `TitleSequence[]` JSON format, allowing the user to load, visually edit, and re-export an existing mod file.

Chapter 4

Simulation Engine Amendments

4.1 4.1 AI Weighting (Marriage & Murder)

- **Marriage:** The engine checks the dynasty's `lowbornSpousesOnly` flag. If true, spouses are generated dynamically as unlanded lowborns. If false, the engine heavily weights marriages toward characters from other active dynasties to simulate alliances, respecting the faith's marital doctrine (monogamy vs. polygamy/concubines). Traits in the `opposites` list act as hard blockers for attraction/marriage.
- **Murder:** If `enableSecrets` is true, characters with high Intrigue or the `deceitful` trait have a chance to generate a `secret_murder`. The target is heavily weighted toward characters defined as a `rival` or `nemesis`, or characters blocking succession.

4.2 4.2 Relationships & Secrets

- **Relationships:** The engine periodically rolls to establish links. *Constraint:* `set_relation_crush` requires age < 16. `set_relation_ward` and `guardian` require a child/adult pairing. `friend`, `nemesis`, `lover`, `soulmate`, and `rival` require both characters to be adults.
- **Secrets:** The engine rolls low-probability events to assign hardcoded secrets (e.g., `secret_deviant`, `secret_witch`, `secret_incest`). `secret_murder` and `secret_lover` require a `target = character:ID` parameter.

4.3 4.3 Cadet Branch Splintering

To prevent massive single-house bloat:

- **Trigger:** If a Dynasty exceeds a threshold of living members, or a member holds a Kingdom-tier title that is NOT the primary Dynasty head's title.
- **Action:** The engine generates a new Cadet House (e.g., `house_U`). The character and all descendants use `dynasty_house = house_U` instead of the base `dynasty`. The new house is appended to the `00_dynasties.txt` output registry.

Chapter 5

Output Formatting Overrides

The following formatting overrides must replace the formatting logic in the existing codebase.

5.1 5.1 Character History Format

The output script must conditionally include relationships, secrets, and the new `dynasty_house` syntax.

```
random_character_id_1 = {
    name = Character_Name

    # If part of the main dynasty:
    # dynasty = dynasty_name
    # If part of a cadet branch:
    dynasty_house = house_name

    religion = faith_name
    culture = culture_name

    female = yes # Must be omitted entirely if male

    father = father_id
    mother = mother_id

    trait = trait_name_1
    trait = trait_name_2

    YYYY.M.D = { birth = yes }

    # Generated ONLY if relationships are rolled
    YYYY.M.D = {
        set_relation_nemesis = character_id_2
        set_relation_lover = character_id_3
    }

    # Generated ONLY if secrets are rolled/enabled
    YYYY.M.D = {
        effect = {
            add_secret = { type = secret_deviant }
            add_secret = { type = secret_murder target = character:
                victim_id }
        }
    }
}
```

```

# Generated ONLY if the character was displaced via Usurpation
YYYY.M.D = { employer = friendly_ruler_id }

YYYY.M.D = {
    death = {
        death_reason = death_execution
        killer = hostile_ruler_id # Omit if death is natural
    }
}
}

```

5.2 5.2 Title History Formats

The engine must format standard landed titles, LAAMPs, and NFs differently. The liege and government properties must be rendered within the date block alongside the holder.

Standard Landed Title:

```

k_gondor = {
    6800.1.1 = {
        holder = house_anarion_ruler_1
        government = feudal_government
        liege = e_gondor
    }
}

```

LAAMP Title (d_laamp_):

```

d_laamp_gildor = {
    700.1.1 = {
        liege = 0
        holder = character_id
        government = landless_adventurer_government
        succession_laws = { landless_adventurer_succession_law }
        effect = {
            if = {
                limit = { NOT = { has_dlc_feature = roads_to_power } }
                holder = { destroy_title = prev }
            }
            create_landless_adventurer_title_history_effect = yes
            set_variable = { name = adventurer_creation_reason value =
                flag:historical }
        }
    }
}

```

Noble Family (NF) Title (d_nf_):

```

d_nf_anarion = {
    3912.1.1 = {
        holder = lineofanarion1
        succession_laws = { noble_family_succession_law }
    }
}

```

5.3 5.3 Dynasty & Cadet House Output

The engine must generate definitions for the main dynasties and any newly splintered cadet houses.

```
dynasty_X = {  
    name = dynn_X  
    culture = some_culture  
    motto = dynn_X_motto  
}  
  
house_U = {  
    name = "dynn_house_U"  
    dynasty = dynasty_X  
    motto = dynn_house_U_motto  
}
```